Artificial Intelligence



Rubik's Cube III (8pts)

Part III: Extending the Problem

In the previous parts, we implemented a solver for 2 × 2 × 2 Rubik's Cube. In this part, we continue with this implementation and extend it to find puzzle solutions for 3 × 3 × 3 Rubik's Cube.

As before, the code for this part should be written in Python to run on tux.cs.drexel.edu, and we will use the same **run.sh** shell script for testing. Again, **you may only use built-in standard libraries (e.g., math, random, time, etc.); you may NOT use any external libraries or packages like Numpy** (if you have any questions at all about what is allowable, please email the instructor).

Terminology

Default state: the solved Rubik's Cube

```
"WWWWWWWWW RRRRRRRRR GGGGGGGGG YYYYYYYYY OOOOOOOOO
BBBBBBBBB"
```

Initial state: a permuted version of the default state (shuffled or scrambled version)

Formulating the Problem (3pts)

Write the code needed to represent a single 3 × 3 × 3 Rubik's Cube state and to compute the possible following cube states after moving a slice. Please start with the provided code (**RubiksCube3.py**) and complete all the functions necessary for solving the problem, same as you did for 2 × 2 × 2 cube. This includes **print()**, **equals()**, **clone()**, **isSolved()**, **applyMove()**, **applyMovesStr()**, and **shuffle()**.

For example:

```
> sh run.sh applyMovesStr "L' B' U' D' R' F' L'" "WWWWWWWWW RRRRRRRRR
GGGGGGGGG YYYYYYYYY OOOOOOOOO BBBBBBBBB"

U L' B' U' D' R'
    WWW                 WWW                 RWW
    WWW                 WWW                 GWW
    WWW                 WWW                 GWW
OOO GGG RRR BBB     GGG RRR BBB OOO     GOO YRR BBB OOW
OOO GGG RRR BBB     OOO GGG RRR BBB     GOO YGG RRR BBW
OOO GGG RRR BBB     OOO GGG RRR BBB     GOO YGG RRR BBW
    YYY                 YYY                 BYY
    YYY                 YYY                 BYY
    YYY                 YYY                 OYY

    GGG                 GWW                 GWW
    GWW                 GWW                 GWW
    GWW                 GGG                 GGG
OOO YRR BBR WWW     WWW OOO YRR BBR     WWW OOO YRR BBR
YOO YGG RRW OBB     YOO YGG RRW OBB     YOO YGG RRW OBB
YOO YGG RRW OBB     YOO YGG RRW OBB     YGG RRW OBB YOO
    BYY                 BYY                 YYB
    BYY                 BYY                 YYR
    RRB                 RRB                 BBR
```

```
    GWY
    GWO
    GGB
WWW OOW RWB RBR
YOO YGW RRB RBB
YGG RRG YRO BOO
    YYO
    YYG
    BBW
```

Bidirectional A* Search (5pts)

Write a method that does a bidirectional A* search from the given cube, returning the first sequence of moves found that reaches the solution state. Similar to regular A*, as part of this process, you will need to choose and implement a heuristic function $h(n)$. You can find a general bidirectional search algorithm in section 3.4.5 of the Russell & Norvig book. Based on the pseudocode, your task is to implement a bidirectional A* search algorithm.

Add a "**bi_astar**" command-line command that allows a user to perform the search. Similar to A*, the output should print the following:

- the entire sequence of moves and relative states to solve the changed cube with **3 cubes** per line,
- the number of nodes that were explored,
- and the time that the function took to find the solution.

Here is an example:

```
> sh run.sh bi_astar "F' F' L' D D U F' L' U' L"

L' U L F U' D D L F F
    GWO                 GWO                 RGG
    YWO                 GWO                 RWW
    YRO                 RRO                 OOO
WGR GWY GGW BBR     RRB WWY GGW BBY     WWY GGW BBY RRB
OOR GGB YRR BBW     GOR YGB YRR BBY     GOR YGB YRR BBY
ORB RYB YOO YGG     WOO WYB YOO YGG     WOO WYB YOO YGG
    WOR                 GOR                 GOR
    YYB                 WYB                 WYB
    WWB                 RWB                 RWB

    GGG                 GGG                 GWW
    YWW                 YWW                 GWW
    BOO                 YWW                 GYY
WGW RGW BBY RRR     WGG ORR BBY RRR     RRR WGG ORR BBY
OOW RGB YRR BBW     OOO YGG ORR BBW     OOO YGG ORR BBW
ORY OYB YOO YGG     ORR BBW OOO YGG     ORR BBW OOO YGG
    GOR                 YYB                 YYB
    YYB                 YYB                 YYB
    WWB                 WWB                 WWB

    GWW                 GWW                 WWW
    GWW                 GWW                 WWW
    GYY                 GYY                 YYY
RRR WGG ORR BBY     RRR WGG ORR BBY     OOR GGG ORR BBB
OOO YGG ORR BBW     OOO YGG ORR BBW     OOR GGG ORR BBB
YGG ORR BBW OOO     OOO YGG ORR BBW     OOR GGG ORR BBB
    WYY                 BWW                 WWW
    WYY                 BYY                 YYY
    BBB                 BYY                 YYY

    WWW                 WWW
```

```
    WWW                 WWW
    RRR                 WWW
OOW GGG YRR BBB     OOO GGG RRR BBB
OOW GGG YRR BBB     OOO GGG RRR BBB
OOW GGG YRR BBB     OOO GGG RRR BBB
    OOO                 YYY
    YYY                 YYY
    YYY                 YYY
1776
0.47
```

Extra Credit (2pts)

For extra points, you can try to improve the run-time of your algorithm as much as possible, and compete against the rest of the class (shorter run-time is better 😉). We might even have a competition between your algorithm and one of the students, with prizes for the winner. For this part, we will check each competitor's timing with a few cubes with solutions at depths of 9 and more. The top 5 agents will receive extra credit up to 2 points. Remember that to have a chance to win the competition, your agent should be able to complete the run in less than a minute, even for solutions with depths greater than 9.

There are a few ways that you can improve the algorithm. The first is to work on improving your data structure and search heuristics. Second, you could implement a different algorithm, like IDA*.

If you would like to enter the competition for extra credit, you have to specifically mention that in your document and add a "competition" command-line command that, just as before, receives a sequence of moves; applies the sequence of moves to the default state $3 \times 3 \times 3$ Rubik's Cube to create an initial state; and then runs the algorithm on it. Again, the output should print the following:

- the entire sequence of moves and relative states to solve the shuffled cube with 3 cubes per line,
- the number of nodes that were explored,
- and the time that the function took to find the solution.

Competition rules: using a database that gives the best heuristic for any cube state disqualifies you for the competition. You also have to describe the heuristic you used in your document.

Academic Honesty

Please remember that you must write all the code for this (and all) assignments by yourself, on your own, without help from anyone except the course TA or instructor.

Submission

Remember that your code must run on **tux.cs.drexel.edu**—that's where we will run the code for testing and grading purposes. Code that doesn't compile or run there will receive a grade of zero.

For this assignment, you must submit:

- Your Python code for this assignment.
- Your **run.sh** shell script that can be run as noted in the examples.
- We should be able to run all the functions mentioned above (apart from the functions for formulating the problem, **bi_astar,** and **competition,** if you would like to enter the competition) using your **run.sh** shell script.
- A PDF document with written documentation containing a few paragraphs

- explaining your program,
- explaining the heuristic you used,
- comparing the time and space complexity of your implementation for bidirectional A* and regular A* that you implemented in the previous part. (one or two paragraphs are enough),
- one paragraph of reasoning about whether bidirectional A* search is optimal or not,
- results showing testing of your routines,
- and if you did anything extra as well as anything that is not working.

Please use a compression utility to compress your files into a single ZIP file (NOT RAR, nor any other compression format). The final ZIP file must be submitted electronically using Blackboard—do not email your assignment to a TA or instructor! If you are having difficulty with your Blackboard account, you are responsible for resolving these problems with a TA or someone from IRT before the assignment is due. If you have any doubts, complete your work early so that someone can help you.